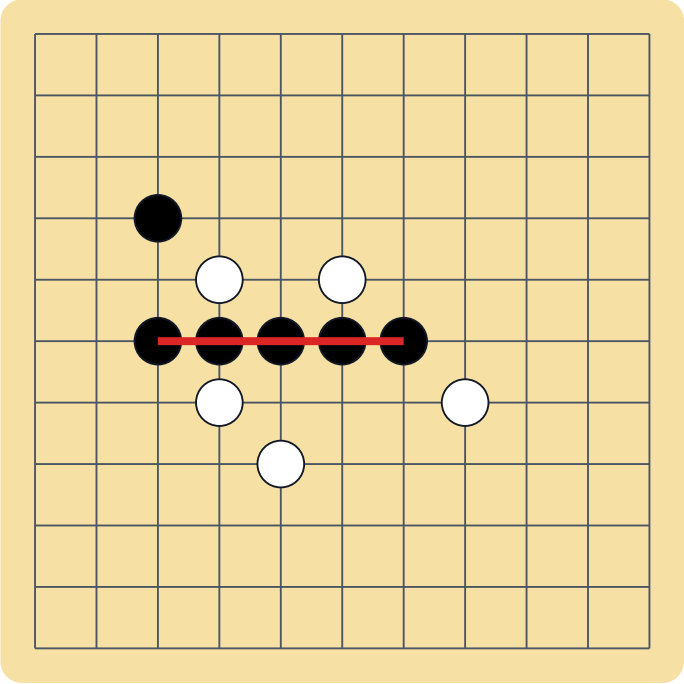
Gomoku

Verbose Rules and Web App Logic

A phone-friendly browser game specification for building a polished five-in-a-row experience.



Example: Black has five connected stones horizontally.

Recommended default for your web app: 15 x 15 freestyle Gomoku, Black moves first, players alternate placing one stone on empty intersections, and the first player to make an unbroken line of five or more stones wins. Add stricter tournament-style rules later as optional modes.

Prepared as an implementation-oriented guide. Date: May 12, 2026.

Contents

This document is written so a developer can use it as a product spec and a rulebook. It intentionally separates the clean beginner version of the game from optional variants that are useful once the basic game is stable.

1. Game summary and recommended default

Gomoku is a two-player abstract strategy game often described as "five in a row." Players take turns placing black and white stones on a grid. Stones do not move after placement in the standard game. The goal is to create a continuous straight line of stones in one of four directions: horizontal, vertical, diagonal down-right, or diagonal down-left.

For a phone-based web app, the best first version is not the most complicated tournament ruleset. The best first version is the one players can understand in ten seconds and replay for a long time. That usually means freestyle Gomoku with a 15 x 15 board, clear touch controls, obvious last-move highlighting, a fast rematch flow, and optional rule toggles for advanced players.

Recommended default ruleset

Rule decision	Recommended default	Why
Board size	15 x 15 intersections	Large enough for real strategy; still manageable on a phone with pinch/zoom or careful spacing.
Players	2 players: Black and White	Keeps the visual language simple and matches the common form of the game.
First move	Black moves first	Common default; easy to explain.
Turn action	Place exactly one stone on one empty intersection	Simple input, simple validation, and no piece movement.
Win condition	Five or more connected stones in a straight line	Freestyle mode is easiest for beginners. Add exact-five as a toggle later.
Draw condition	Board is full and no player has won	Clear ending when all 225 intersections are occupied on a 15 x 15 board.
Undo	Off by default for ranked/online; optional for local casual games	Prevents disputes and cheating in competitive play.

Product advice: launch with one clean mode, then add variants. A game that starts instantly and teaches itself will outperform a rules-heavy game that makes the player choose among variants before they understand the base game.

Why it is deeper than normal tic tac toe

- The board is much larger, so the game has many more possible positions and strategic paths.
- A player can create threats that force the opponent to respond, then chain those threats into a win.
- Forks matter. A player can make a move that creates two possible winning lines at once, leaving the opponent unable to block both.
- Defense is not just blocking the current line; good defense stops future open-threes and open-fours before they become forced wins.
- Because stones stay on the board, every move permanently changes the tactical landscape.

2. Player-facing rules

These are the rules you can show in an in-game "How to Play" screen. They are intentionally written in plain language for players, not programmers.

Object of the game

Be the first player to place five of your stones in a continuous straight line. The line may be horizontal, vertical, or diagonal. In the recommended freestyle mode, a line of six or more also counts as a win because it contains at least five connected stones.

Setup

1. The game uses a square grid. The recommended board is 15 x 15 intersections.
2. One player uses black stones. The other player uses white stones.
3. Black takes the first turn unless a selected variant says otherwise.
4. The board starts empty.

Taking a turn

1. On your turn, tap an empty intersection on the board.
2. Your stone is placed on that intersection.
3. After the stone is placed, the game checks whether your move created a winning line.
4. If you did not win and the board is not full, the turn passes to the other player.

Legal moves

A move is legal when all of the following are true:

- The game is currently active.
- It is that player's turn.
- The selected row and column are inside the board.
- The selected intersection is empty.
- The move does not violate the selected rule variant, such as exact-five or Renju-style forbidden moves.

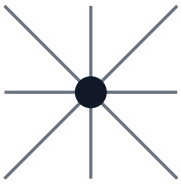
Illegal moves

A move is illegal if a player taps outside the board, taps an occupied intersection, tries to move when it is not their turn, or tries to place a stone after the game has ended. The app should reject the move and show a small, calm message such as "That spot is already taken" or "Wait for your turn."

Winning

A player wins immediately after placing a stone that completes a continuous straight line of at least five of their own stones. The winning line can be in any of these four direction pairs:

Only four direction pairs need checking after each move



horizontal: (0, 1)

vertical: (1, 0)

diagonal down-right: (1, 1)

diagonal down-left: (1, -1)

Drawing

The game is a draw if every intersection is filled and neither player has a winning line. On a 15 x 15 board, that means all 225 intersections are occupied. The app should show "Draw" and offer a rematch.

Resignation and timeout

For a polished web app, you may add two non-board endings: resignation and timeout. A resigned player loses immediately. A player who runs out of time loses immediately if timed games are enabled. These are app features, not necessary for the core rules.

3. Board, coordinates, and terminology

Intersections, not cells

A key design detail: Gomoku stones are placed on grid intersections, not inside square cells. This matters for drawing and touch detection. A 15 x 15 board has 15 horizontal grid lines and 15 vertical grid lines, producing 225 playable intersections. Visually, the stone center should sit exactly on a line crossing.

Coordinate system

For code, use zero-based coordinates. The top-left intersection is row 0, column 0. The bottom-right intersection on a 15 x 15 board is row 14, column 14. The UI may display friendlier coordinates if desired, but the internal model should stay consistent.

Term	Meaning in app logic
row	Vertical index from top to bottom. Valid range: 0 to size - 1.
col	Horizontal index from left to right. Valid range: 0 to size - 1.
board[row][col]	The value at one intersection: empty, black, or white.
lastMove	The most recent stone placed. Win detection only needs to check lines through this point.
direction	A row/column step pair such as (0, 1), (1, 0), (1, 1), or (1, -1).

Important terms

Term	Definition
Stone	A black or white marker placed on an empty intersection.
Line	A continuous sequence of same-color stones in one straight direction.
Five	Exactly five connected stones in a row.

Term	Definition
Five-or-more	Five, six, seven, or more connected stones. This wins in freestyle mode.
Overline	A line longer than five. Some variants count it as a win; exact-five and some tournament rules do not.
Open end	An empty intersection immediately beyond one end of a line.
Closed end	An end blocked by the board edge or the opponent's stone.
Open three	A line of three stones with space to extend on both ends. This is a common tactical threat.
Open four	A line of four stones with two possible winning ends. This is usually an urgent threat.

Recommended visual coordinate mapping

For canvas or SVG rendering, do not divide the board into 15 cell widths. Because stones are on intersections, divide the inner board width by size - 1. For a 15 x 15 board, there are 14 spaces between 15 intersections.

```
innerSize = boardPixelSize - 2 * padding
spacing = innerSize / (boardSize - 1)

xForCol(col) = padding + col * spacing
yForRow(row) = padding + row * spacing

colFromTouch(x) = round((x - padding) / spacing)
rowFromTouch(y) = round((y - padding) / spacing)
```

After rounding, validate that row and col are inside the board and that the touch is close enough to an intersection. A distance threshold of about 0.45 x spacing feels good on phones. This prevents accidental placements from taps between intersections.

4. Turn lifecycle and game states

A clean app should treat every move as a small transaction: validate the input, apply the move, check the ending condition, then update the UI. Do not let UI rendering and rules logic become mixed together. Keep the rules engine pure and testable.

Game states

State	Meaning	Allowed actions
setup	Players are choosing mode, board size, color, timer, or opponent type.	Start game, change settings.
playing	The board is active and a player can move.	Place stone, resign, offer draw, open menu.
paused	Game is temporarily suspended, usually local-only.	Resume, restart, exit.
ended	The game has a winner, draw, resignation, or timeout.	Review board, rematch, share result.

Move lifecycle

1. Receive input from a tap, click, keyboard, AI, or network message.
2. Convert input to a board coordinate: row and col.
3. Check that the game status is playing.

4 Check that the acting player equals currentPlayer.

5 Check that row and col are inside the board.

6 Check that board[row][col] is empty.

7 Check any selected variant restrictions.

8 Place the stone and append the move to moveHistory.

9 Run win detection from the new stone.

10 If the move wins, set status to ended and store the winning line.

11 If the board is full and there is no win, set status to ended with draw.

12 Otherwise switch currentPlayer and wait for the next move.

State transition diagram in words

setup -> playing when a new game starts. playing -> ended when a player wins, the board fills, a player resigns, or a timer expires. playing -> paused only for local games where pausing is allowed. paused -> playing when the player resumes. ended -> setup or playing depending on whether the rematch uses a settings screen or starts instantly.

Move history

Keep a move history even if you do not support undo. It is useful for replay, debugging, analytics, multiplayer reconciliation, and detecting suspicious network messages. A move record should include move number, player, row, col, timestamp, and possibly clientId or serverSequence for online games.

```
Move = {
  index: 37,
  player: "BLACK",
  row: 7,
  col: 8,
  createdAt: "2026-05-12T18:42:19.000Z",
  source: "local_touch"  // local_touch, remote, ai, replay
}
```

5. Win detection and draw detection

Core idea

After a stone is placed, you do not need to scan the whole board. Only the newest stone can create a new winning line. Check the four direction pairs that pass through the new stone. Count matching stones forward and backward in each direction. If the total count reaches the selected win threshold, the move wins.

The four directions

Direction pair	Delta	Example line
Horizontal	(0, 1) and (0, -1)	left to right
Vertical	(1, 0) and (-1, 0)	top to bottom
Diagonal down-right	(1, 1) and (-1, -1)	top-left to bottom-right
Diagonal down-left	(1, -1) and (-1, 1)	top-right to bottom-left

Counting line length

For each direction pair, count consecutive same-color stones in the positive direction and the negative direction. Add 1 for the newly placed stone. For example, if the player has two stones to the left, two stones to the right, and the new stone in

the center, the total is 2 + 1 + 2 = 5, so the player wins.

Freestyle vs exact-five

Mode	How a line wins	Overline behavior
Freestyle	Line length >= 5	A line of 6 or more wins because it includes five connected stones.
Exact-five	Line length == 5	A line of 6 or more does not count as a win. The app should call this an overline.
Renju-style	More advanced and asymmetric	Black has special restrictions; not recommended as the first ruleset.

Winning line coordinates

When a move wins, store the exact endpoints of the winning segment. This lets the UI draw a red line, glow, or animation over the winning stones. In freestyle mode, if a player makes six or more in a row, you can highlight the full connected line or just the first five containing the new stone. Highlighting the full connected line is clearer.

Draw detection

The simplest draw check is a move counter. A 15 x 15 board has 225 possible moves. Increment occupiedCount after every legal placement. If occupiedCount equals boardSize x boardSize and no win was found on the last move, the game is a draw.

```
if winningLine exists:
  status = "ended"
  result = { type: "win", winner: currentPlayer, line: winningLine }
else if occupiedCount == boardSize * boardSize:
  status = "ended"
  result = { type: "draw" }
else:
  currentPlayer = opposite(currentPlayer)
```

Performance

This algorithm is effectively constant-time for normal board sizes. On a 15 x 15 board, each direction can count at most 14 intersections each way. That is tiny. You can run it after every move without performance concerns, even on low-end phones.

6. Web app data model

Keep the game model small, explicit, and serializable. A good rule of thumb: the full state should be easy to store as JSON, send over a WebSocket, save in localStorage, or replay from a database.

Recommended TypeScript-style model

```
type Player = "BLACK" | "WHITE";
type Cell = "EMPTY" | Player;
type GameStatus = "setup" | "playing" | "paused" | "ended";
type RuleMode = "freestyle" | "exact_five" | "swap2";

type Move = {
  index: number;
  player: Player;
  row: number;
  col: number;
  createdAt: string;
};

type WinningLine = {
  player: Player;
```

```
  start: { row: number; col: number };
  end: { row: number; col: number };
  direction: { dr: number; dc: number };
  length: number;
};

type GameState = {
  id: string;
  boardSize: number;
  board: Cell[][];
  status: GameStatus;
  ruleMode: RuleMode;
  currentPlayer: Player;
  occupiedCount: number;
  moveHistory: Move[];
  lastMove?: Move;
  winningLine?: WinningLine;
  result?:
    | { type: "win"; winner: Player; reason: "five_in_row" | "timeout" | "resign" }
    | { type: "draw" };
};
```

Board representation choices

Representation	Pros	Cons
2D array	Easy to understand: board[row][col]. Best for first implementation.	JSON is a little larger than a compact string.
Flat array	Fast and compact: cells[row * size + col].	Less readable; easy to make index mistakes.
Move list only	Small storage and easy replay.	Must reconstruct board for validation unless cached.
Bitboards	Very fast for AI and advanced search.	Overkill for a normal web app.

Recommended storage strategy

- Local casual game: store GameState in memory and optionally mirror it to localStorage for recovery after refresh.
- Online game: treat the server as authoritative. Clients send move requests; the server validates and broadcasts accepted moves.
- Replay: store the initial settings and moveHistory. Reconstruct board from the move list when replaying.
- Analytics: store result, move count, duration, board size, and rule mode, but avoid collecting more player data than needed.

Do not store UI state inside the rules engine

Zoom level, selected theme, animation progress, hovered intersection, sound settings, and modal visibility should live outside the core GameState. This prevents animation bugs from corrupting the actual game rules.

7. Pseudocode for core logic

The following pseudocode is deliberately close to JavaScript or TypeScript, but it is written to explain the logic rather than force a particular framework.

Create a new game

```
function createGame(boardSize = 15, ruleMode = "freestyle") {
  return {
    id: generateId(),
    boardSize,
    board: Array.from({ length: boardSize }, () =>
      Array.from({ length: boardSize }, () => "EMPTY")
    ),
    status: "playing",
    ruleMode,
    currentPlayer: "BLACK",
    occupiedCount: 0,
    moveHistory: []
  };
}
```

Validate a move

```
function validateMove(game, player, row, col) {
  if (game.status !== "playing") {
    return { ok: false, reason: "Game is not active." };
  }
  if (player !== game.currentPlayer) {
    return { ok: false, reason: "It is not this player's turn." };
  }
  if (!isInsideBoard(game.boardSize, row, col)) {
    return { ok: false, reason: "Move is outside the board." };
  }
  if (game.board[row][col] !== "EMPTY") {
    return { ok: false, reason: "That intersection is already occupied." };
  }
  if (violatesVariantRule(game, player, row, col)) {
    return { ok: false, reason: "Move is forbidden by this rule mode." };
  }
  return { ok: true };
}
```

Apply a move

```
function applyMove(game, player, row, col) {
  const validation = validateMove(game, player, row, col);
  if (!validation.ok) return { game, accepted: false, reason: validation.reason };

  const next = cloneGame(game);
  next.board[row][col] = player;
  next.occupiedCount += 1;

  const move = {
    index: next.moveHistory.length + 1,
    player,
    row,
    col,
    createdAt: new Date().toISOString()
  };
  next.moveHistory.push(move);
  next.lastMove = move;

  const line = findWinningLine(next.board, row, col, player, next.ruleMode);

  if (line) {
    next.status = "ended";
    next.winningLine = line;
    next.result = { type: "win", winner: player, reason: "five_in_row" };
  } else if (next.occupiedCount === next.boardSize * next.boardSize) {
    next.status = "ended";
    next.result = { type: "draw" };
  } else {
```

```
    next.currentPlayer = player === "BLACK" ? "WHITE" : "BLACK";
  }

  return { game: next, accepted: true };
}
```

7. Pseudocode for core logic, continued

Count stones in one direction

```
function countDirection(board, row, col, dr, dc, player) {
  let count = 0;
  let r = row + dr;
  let c = col + dc;

  while (isInsideBoard(board.length, r, c) && board[r][c] === player) {
    count += 1;
    r += dr;
    c += dc;
  }

  // Also return the first non-matching coordinate so the caller can compute
  // endpoints and open/closed ends if needed.
  return { count, stoppedAt: { row: r, col: c } };
}
```

Find a winning line

```
function findWinningLine(board, row, col, player, ruleMode) {
  const directions = [
    { dr: 0, dc: 1 },   // horizontal
    { dr: 1, dc: 0 },   // vertical
    { dr: 1, dc: 1 },   // diagonal down-right
    { dr: 1, dc: -1 }   // diagonal down-left
  ];

  for (const { dr, dc } of directions) {
    const forward = countDirection(board, row, col, dr, dc, player);
    const backward = countDirection(board, row, col, -dr, -dc, player);
    const length = 1 + forward.count + backward.count;

    const winsFreestyle = ruleMode === "freestyle" && length >= 5;
    const winsExactFive = ruleMode === "exact_five" && length === 5;

    if (winsFreestyle || winsExactFive) {
      return {
        player,
        start: { row: row - backward.count * dr, col: col - backward.count * dc },
        end: { row: row + forward.count * dr, col: col + forward.count * dc },
        direction: { dr, dc },
        length
      };
    }
  }

  return null;
}
```

Touch-to-intersection conversion

```
function touchToIntersection(x, y, boardRect, boardSize, padding) {
  const inner = boardRect.width - 2 * padding;
  const spacing = inner / (boardSize - 1);

  const localX = x - boardRect.left - padding;
  const localY = y - boardRect.top - padding;

  const col = Math.round(localX / spacing);
  const row = Math.round(localY / spacing);

  if (!isInsideBoard(boardSize, row, col)) return null;

  const snappedX = padding + col * spacing;
```

```
  const snappedY = padding + row * spacing;
  const dx = localX - snappedX;
  const dy = localY - snappedY;
  const distance = Math.sqrt(dx * dx + dy * dy);

  if (distance > spacing * 0.45) return null;
  return { row, col };
}
```

8. Phone UI behavior and interaction design

The rules can be perfect and the game can still feel bad if the phone interaction is cramped. Gomoku has a large board, so the app needs careful input design.

Recommended screen layout

Area	Purpose	Notes
Top bar	Shows player turn, timer, and menu.	Use clear black/white labels and a small last-move indicator.
Board	Main interactive play area.	Make it as large as possible. Square, centered, with safe padding.
Action row	Undo, resign, rules, rematch, or confirm.	Keep dangerous actions behind confirmation.
Toast/message	Shows invalid move messages.	Short, non-blocking, and dismisses automatically.

Input patterns

- Direct tap: fastest. Tap an empty intersection to place immediately. Best for casual play when board spacing is comfortable.
- Tap then confirm: safer. First tap previews the stone, then player taps a confirm button. Best for small screens or ranked online play.
- Zoom and pan: useful on very small screens, but can slow the game. Use only if direct tap feels inaccurate.
- Magnifier loupe: show a zoomed preview near the finger while dragging. This helps precision without full-board zoom.

Visual feedback

- Highlight the current player clearly before every move.
- Mark the last move with a small dot or ring so players can follow the game.
- Preview the selected intersection before committing a move in confirm mode.
- Animate stone placement subtly, but do not slow down repeated play.
- When someone wins, draw a line through the winning stones and freeze the board.
- Use color plus shape or outline so the game is readable for color-blind players and in low brightness.

Board sizing formula

A good mobile board tries to use the smaller of viewport width and available viewport height. Leave room for a turn label and action buttons. If the board becomes too small, switch to confirm mode automatically.

```
availableHeight = viewportHeight - topBarHeight - actionBarHeight - safeAreaPadding
boardPixelSize = min(viewportWidth - 24, availableHeight)

if boardPixelSize / (boardSize - 1) < 22 pixels:
  enableTapPreviewOrZoom = true
```

Accessibility

Support keyboard input for desktop users, screen reader labels for game status, sufficient contrast, and reduced-motion mode. For screen readers, announce moves like "Black placed at row 8, column 9. White to move." Do not rely only on color to communicate which stones are black or white.

9. Multiplayer, sync, and anti-cheat logic

Gomoku is excellent for both local pass-and-play and online multiplayer. The rules are deterministic, the messages are small, and the server can validate every move cheaply.

Local pass-and-play

The simplest version runs fully in the browser. Both players use the same device and alternate turns. This mode should support instant rematch and optional undo. It does not need accounts, servers, or matchmaking.

Online multiplayer architecture

1. Client A taps a move and sends { gameId, playerId, row, col, clientMoveId } to the server.
2. The server loads the authoritative game state.
3. The server validates the move using the same rules engine.
4. If valid, the server appends the move, updates the state, assigns a server sequence number, and broadcasts the accepted move to both clients.
5. If invalid, the server rejects it and tells the client to resync.

Why the server should be authoritative

Never let an online client declare that it won. The client can suggest a move, but the server should decide whether that move is legal and whether it wins. This prevents modified clients from placing stones on occupied intersections, moving twice, changing colors, or claiming a false result.

Race conditions

Problem	Example	Fix
Double submit	Player taps twice quickly.	Disable board after first pending move; use clientMoveId for idempotency.
Out-of-order messages	Client receives move 42 before move 41.	Apply moves only by serverSequence; request resync if a gap appears.
Two clients think it is their turn	Reconnect after stale state.	Server rejects any player not equal to currentPlayer.
Refresh during game	Player reloads tab.	Fetch authoritative state by gameId and replay moveHistory.

Suggested network events

```
client -> server: join_game       { gameId, playerToken }
server -> client: game_snapshot   { gameState, serverSequence }
client -> server: submit_move     { gameId, row, col, clientMoveId }
server -> clients: move_accepted  { move, gameStatePatch, serverSequence }
server -> client: move_rejected   { clientMoveId, reason, latestSnapshot }
server -> clients: game_ended     { result, winningLine }
```

Cheat-resistant timers

If timed play matters, the server should own the clocks. Clients can display local countdowns for smoothness, but the server should calculate timeout based on server time and accepted move timestamps.

10. Optional AI opponent

An AI opponent is optional, but it can make the game much more useful when no human opponent is available. You do not need a perfect Gomoku AI. A simple tactical AI can still feel good for casual players.

AI levels

Level	Behavior	Implementation difficulty
Beginner	Random legal moves, but blocks immediate five-in-a-row threats.	Low
Casual	Scores candidate moves by creating or blocking twos, threes, and fours.	Medium
Strong	Uses minimax with alpha-beta pruning and threat evaluation.	High
Expert	Uses advanced threat-space search and opening knowledge.	Very high

Candidate move reduction

Instead of evaluating every empty intersection, only consider empty intersections near existing stones. For example, collect all empty points within two spaces of any stone. This makes the AI faster and also makes its moves feel more natural.

```
function getCandidateMoves(board, radius = 2) {
  const candidates = new Set();

  for each occupied cell (r, c):
    for dr from -radius to radius:
      for dc from -radius to radius:
        nr = r + dr
        nc = c + dc
        if inside board and board[nr][nc] is EMPTY:
          candidates.add(`${nr},${nc}`)

  if candidates is empty:
    return center point

  return candidates as coordinate list
}
```

Simple scoring priorities

1. If the AI can win immediately, play that move.
2. If the opponent can win immediately, block that move.
3. Prefer moves that create an open four.
4. Block opponent open fours.
5. Prefer moves that create open threes or multiple threats.
6. Block opponent open threes if they are dangerous.
7. Prefer center and moves close to friendly stones when no tactics are urgent.

Do not block the UI

Run heavier AI thinking in a Web Worker so the board does not freeze. For a casual phone web app, cap thinking time. A fast, responsive AI is usually better than a stronger AI that makes the interface feel broken.

11. Rule variants and balancing options

Gomoku has a known first-player advantage in unrestricted play. For casual play this may be acceptable, especially if players alternate who starts. For more serious play, offer balancing variants as explicit options rather than hiding them inside the default rules.

Variant comparison

Variant	Description	Best use
Freestyle	Five or more in a row wins. No forbidden moves.	Default casual mode and fastest onboarding.
Exact-five	Exactly five wins; six or more is an overline and does not count.	Players who want stricter rules but not full Renju complexity.
Pro opening	Black starts in center; Black's second stone must be a minimum distance away.	Simple tournament-inspired balancing.
Swap	First player creates an opening position; second player chooses which color to play.	Balanced games between experienced players.
Swap2	First player places two black and one white stone; second player may choose color or add two more stones and pass color choice back.	Advanced competitive mode.
Renju-style	Adds forbidden moves for Black, such as double-three and double-four restrictions.	Advanced players; not ideal as first version.

How to expose variants in the app

Do not overwhelm new players with every variant on the home screen. Use a default "Play" button for freestyle. Put variants behind "Game Options" or "Advanced Rules." Always show the chosen rule mode during the game so both players know how overlines and special openings are handled.

Implementing exact-five

Exact-five only changes the win check. Instead of accepting length >= 5, require length == 5. If a player makes six or more, the result depends on your variant design. The simplest exact-five app rule is: overline is not a win, but the move remains on the board and play continues. Make that rule clear before the game starts.

Implementing Swap2 at a high level

1. Tentative first player places three stones total: two black stones and one white stone anywhere on the board.
2. Tentative second player chooses one of three options: play as White, swap and play as Black, or place two more stones and pass the color choice back.
3. If the second player places two more stones, they place one black and one white stone.
4. The original tentative first player then chooses which color to play.
5. After colors are assigned, normal play continues from the current board position with the correct side to move.

This opening protocol is powerful but not beginner-friendly. Treat it as an advanced option after the base app is polished.

12. Edge cases, test cases, and implementation checklist

Core edge cases

Edge case	Expected behavior
Tap occupied point	Reject move; do not switch turns.
Tap outside board	Ignore or show a light invalid tap message.
Win on board edge	Win should still count; board edge does not block a completed five.
Diagonal anti-slash win	Check direction (1, -1); do not only check main diagonal.
Overline in freestyle	Count as a win.
Overline in exact-five	Do not count as a win unless your variant says otherwise.
Final move fills board and wins	Win beats draw because the last move created a line.
Move after game ended	Reject move and keep final board unchanged.
Refresh during online game	Reload authoritative game state and current turn.

Test cases for win detection

Test	Example	Expected result
Horizontal five	Black at (7,3), (7,4), (7,5), (7,6), then (7,7).	Black wins.
Vertical five	White at (2,10), (3,10), (4,10), (5,10), then (6,10).	White wins.
Diagonal down-right	Black at (4,4), (5,5), (6,6), (7,7), then (8,8).	Black wins.
Diagonal down-left	White at (4,10), (5,9), (6,8), (7,7), then (8,6).	White wins.
Gap in line	Black at (7,3), (7,4), empty (7,5), (7,6), (7,7).	No win.
Split by opponent	Black at (7,3), (7,4), White at (7,5), Black at (7,6), (7,7).	No win.
Six in freestyle	Black forms six connected stones.	Black wins.
Six in exact-five	Black forms six connected stones.	No win by exact-five rule.

Implementation checklist

- Rules engine is separate from UI components.
- Board uses a clear row/col coordinate system.
- Touch input snaps to intersections, not square centers.
- Move validation rejects occupied, out-of-turn, out-of-bounds, and post-game moves.
- Win detection checks all four direction pairs from the last move.
- App stores winning line endpoints for UI highlighting.
- Draw detection uses occupiedCount after checking for a win.
- Game result is immutable once status becomes ended.
- Online mode uses server-authoritative validation.

- In-game rules screen clearly states whether overlines count.
- Local casual mode has fast rematch.
- Advanced variants are optional and clearly labeled.

Minimum viable version

The minimum polished version is: local two-player freestyle Gomoku, 15 x 15 board, tap-to-place or tap-confirm input, last-move marker, win-line highlight, draw detection, reset/rematch button, and a concise rules modal. Build that first. Then consider AI, online play, timers, accounts, ratings, and advanced variants.

13. Sources consulted

This guide uses the common beginner-friendly form of Gomoku as the recommended app default and references public rule descriptions for terminology and variants. The implementation guidance, app architecture, pseudocode, UI recommendations, and test checklist are original synthesis for a web-based phone game.

Source	Used for
Wikipedia, "Gomoku" - https://en.wikipedia.org/wiki/Gomoku	Common overview: two-player game, 15 x 15 board, alternating placement, Black first, five-in-a-row win, draw, variants, and first-player advantage.
RenjuNet, "Renju Rules" - https://gomoku.renju.net/rule/	Official RenjuNet rules index for Gomoku and Renju-related variants.
RenjuNet, "Gomoku - Swap 2 Rule" - https://gomoku.renju.net/rule/11/	Swap2 opening summary: first player places two black and one white stone; second player chooses among color and placement options.

Bottom line: make the first version easy to start, hard to master, and unambiguous about overlines. The rules engine should be tiny, testable, and independent of the UI. That gives you a strong foundation for online play, AI, and advanced rule variants later.